

BigInteger contra OpenJDK

Qué es el fuzzing

El fuzzing es una forma de probar software sin escribir los casos a mano. Un programa genera entradas en cantidad, las ejecuta y vigila que no pase nada que no debería. Nació buscando cuelgues con datos al azar, y hoy es una de las técnicas que más errores encuentra en compiladores, navegadores, parsers y bibliotecas.

Generar entradas es lo fácil. Lo difícil es **saber si una respuesta está mal sin conocer la correcta**. A eso se le llama el problema del oráculo, y hay tres formas habituales de resolverlo:

Que no se rompa

Solo se vigilan cuelgues, errores inesperados o memoria corrupta. No hace falta saber nada del resultado.

Propiedades

Algo que vale siempre: decodificar lo codificado devuelve la entrada, ordenar dos veces da lo mismo que ordenar una.

Diferencial el de este informe

Dos implementaciones de lo mismo tienen que coincidir. Cualquier diferencia es un error de una de las dos.

Dos técnicas lo vuelven práctico. **Generar con sesgo hacia los bordes**, porque los errores se concentran en ceros, negativos y límites de tamaño. Y **reducir** cada falla a la entrada más chica que todavía la reproduce, para que una persona la entienda de un vistazo.

Por qué vale la pena

- **No hay que escribir resultados esperados.**
La versión de referencia cumple ese papel, así que probar mil entradas cuesta lo mismo que probar diez.
- **Encuentra lo que a nadie se le ocurre probar a mano.**
Negativos, ceros, límites de 32 y 64 bits, números enormes: los casos que un test escrito a mano suele saltar.
- **Todo se reproduce desde una semilla.**
Cualquier hallazgo se puede volver a mostrar, en otra máquina y otro día.
- **Entrega el caso más chico que falla.**
La entrada que dispara un error suele ser larga y estar llena de detalles que no tienen nada que ver. El fuzzer la achica sola, quitando pedazos mientras el error siga apareciendo, y quien lo arregla ve directamente dónde está el problema.

Cómo implementamos el fuzzer

El proyecto ya tenía un fuzzer, escrito en Rust, que prueba la máquina virtual y el compilador con programas generados. Para la biblioteca escribimos uno más chico, en Python: `test-fuzzing/fuzzer/bigfuzz.py`. Se corre a mano, guarda todo lo que mide en `test-fuzzing/datos/`, y esta página solo muestra esos datos.

1. **Generar.** Arma al azar expresiones de `BigInteger` como árboles de operaciones, de hasta tres niveles, y las escribe como código Java. Nadie las escribe a mano: las operaciones y cómo se anidan salen al azar, y las constantes salen, seis de cada diez veces, de una lista fija de valores límite. Todo parte de una semilla, así que la misma semilla arma siempre las mismas expresiones.
2. **Ejecutar.** Junta 120 de las expresiones generadas en una sola clase, cada una en su propio método con un `try/catch` que devuelve el resultado o el nombre de la excepción. La compila una vez con el `javac` de OpenJDK y corre el mismo `.class` en OpenJDK y en KajiJDK.
3. **Comparar.** Lee lo que imprimió cada lado, línea por línea, y compara el texto de cada resultado.
4. **Reducir.** Para cada tipo de divergencia arma versiones más chicas de la expresión, las corre todas juntas en un solo programa y se queda con la más chica que sigue divergiendo. Repite hasta que no puede achicar más.

De dónde sale el azar. Del generador pseudoaleatorio de Python, `random.Random`, que usa el algoritmo Mersenne Twister y arranca desde la semilla que recibe el script con `--seed`. No es

azar verdadero: es una secuencia de números que parece aleatoria pero que la semilla deja fija. Cada decisión del generador toma el número siguiente de esa secuencia: qué operación usar, si una constante sale de la lista de límites o se inventa, cuántos dígitos tiene, de 1 a 60, y si es negativa. Por eso la misma semilla repite la campaña entera, y otra semilla prueba expresiones distintas.

La prueba

El fuzzer generó expresiones de `java.math.BigInteger` con valores elegidos hacia los bordes, y las corrió en **Eclipse Adoptium 25.0.4.1** y en **KajijDK con KajiLibrary**. O sea, se llamó a dos clases que deberían dar el mismo resultado, el `BigInteger` de OpenJDK y el de KajiLibrary, y se las probó con entradas aleatorias. Después comparó los resultados uno por uno y redujo cada divergencia distinta a su caso más chico.

Campaña terminada Referencia **Eclipse Adoptium 25.0.4.1** Bajo prueba **KajijDK (run-headless) con KajiLibrary**

Semilla **1** · 6 lotes de 120 17 de septiembre de 2026 a las 12:29 p. m.

720 expresiones comparadas	704 dieron lo mismo	16 divergieron	2 errores distintos en KajiLibrary	8 casos mínimos
16,2 min de campaña				

otro valor: 15 **excepción contra valor: 1**

Qué es una divergencia

Una **divergencia** es una expresión en la que OpenJDK y KajijDK devolvieron resultados distintos. Por ejemplo, `new BigInteger("-1").shiftRight(1).toString()` da `-1` en OpenJDK y `0` en KajijDK. Como OpenJDK hace de referencia, cada divergencia indica que KajiLibrary se aparta de lo que tendría que hacer.

Las etiquetas de arriba dicen de qué forma difieren:

- otro valor:** las dos devolvieron un resultado, pero distinto.
- excepción contra valor:** una lanzó una excepción y la otra devolvió un resultado.

Una divergencia no es lo mismo que un error: varias pueden salir del mismo error, con entradas distintas. Las 16 divergencias de esta campaña se explican con 2 errores, que se detallan abajo.

Qué encontró

El reductor achicó cada divergencia distinta hasta su caso mínimo, y varias quedaron en la misma expresión. Acá están agrupadas por causa. Los números salen de la campaña; la explicación de cada causa se escribió a mano, después de leer el código de las dos implementaciones y verificarla en OpenJDK.

shiftRight redondea hacia cero con los negativos Error de KajiLibrary

La especificación dice que `shiftRight(n)` calcula `floor(this / 2^n)`: redondea hacia menos infinito, igual que `>>` sobre un `int`. KajiLibrary desplaza la magnitud y conserva el signo, y así trunca hacia cero: `-1 >> 1` da 0 en vez de `-1`. OpenJDK corrige ese caso sumando uno a la magnitud cuando el número es negativo y el desplazamiento pierde algún bit en 1.

KajiLibrary: `java/math/BigInteger.java`, `shiftRight`, líneas 522 a 527. OpenJDK: `shiftRightImpl`.

```
new BigInteger("-1").shiftRight(1).toString()
```

OPENJDK	KAJIJDk
-1	0

3 divergencias de la campaña se redujeron a esta expresión.

Cómo se redujo: de 63 a 45 caracteres en 2 pasos

Cómo se redujo: de 78 a 45 caracteres en 3 pasos

Cómo se redujo: de 191 a 45 caracteres en 4 pasos

```
new BigInteger("-2").shiftRight(64).toString()
```

OPENJDK	KAJIJDk
-1	0

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 184 a 46 caracteres en 2 pasos

testBit ignora el signo Error de KajiLibrary

La especificación dice que `testBit(n)` responde sobre la representación en complemento a dos, la misma que usan `and`, `or` y `not`. En complemento a dos `-1` tiene todos los bits en 1, así que `testBit(31)` es `true`. KajiLibrary lee el bit de la magnitud sin mirar el signo: la magnitud de `-1` es 1, que no tiene el bit 31 encendido, y responde `false`.

KajiLibrary: `java/math/BigInteger.java`, `testBit`, líneas 508 a 513, y `testBitMag`, líneas 270 a 276.

```
String.valueOf(new BigInteger("-2").testBit(31))
```

OPENJDK

true

KAJIJDk

false

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 141 a 48 caracteres en 3 pasos

```
String.valueOf(new BigInteger("-1").testBit(31))
```

OPENJDK

true

KAJIJDk

false

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 89 a 48 caracteres en 1 pasos

```
String.valueOf(new BigInteger("1").negate().testBit(63))
```

OPENJDK

true

KAJIJDk

false

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 58 a 56 caracteres en 1 pasos

modPow lanza una excepción, por culpa de shiftRight Consecuencia de otro error

Por el error de `shiftRight`, KajiLibrary calcula `-1 >> 1` como 0, y 0 no tiene inverso módulo 2: un `modPow` con exponente negativo tiene que lanzar. OpenJDK hace exactamente lo mismo si recibe un 0, porque `BigInteger.ZERO.modPow(-2, 2)` también lanza `ArithmeticException`. `modPow` se comporta bien; el valor equivocado llega de antes.

Se arregla con `shiftRight`.

```
new BigInteger("-1").shiftRight(1).modPow(new BigInteger("2").negate(), new BigInteger("2")).toString()
```

OPENJDK

1

KAJIJDk

EXC: java.lang.ArithmeticException

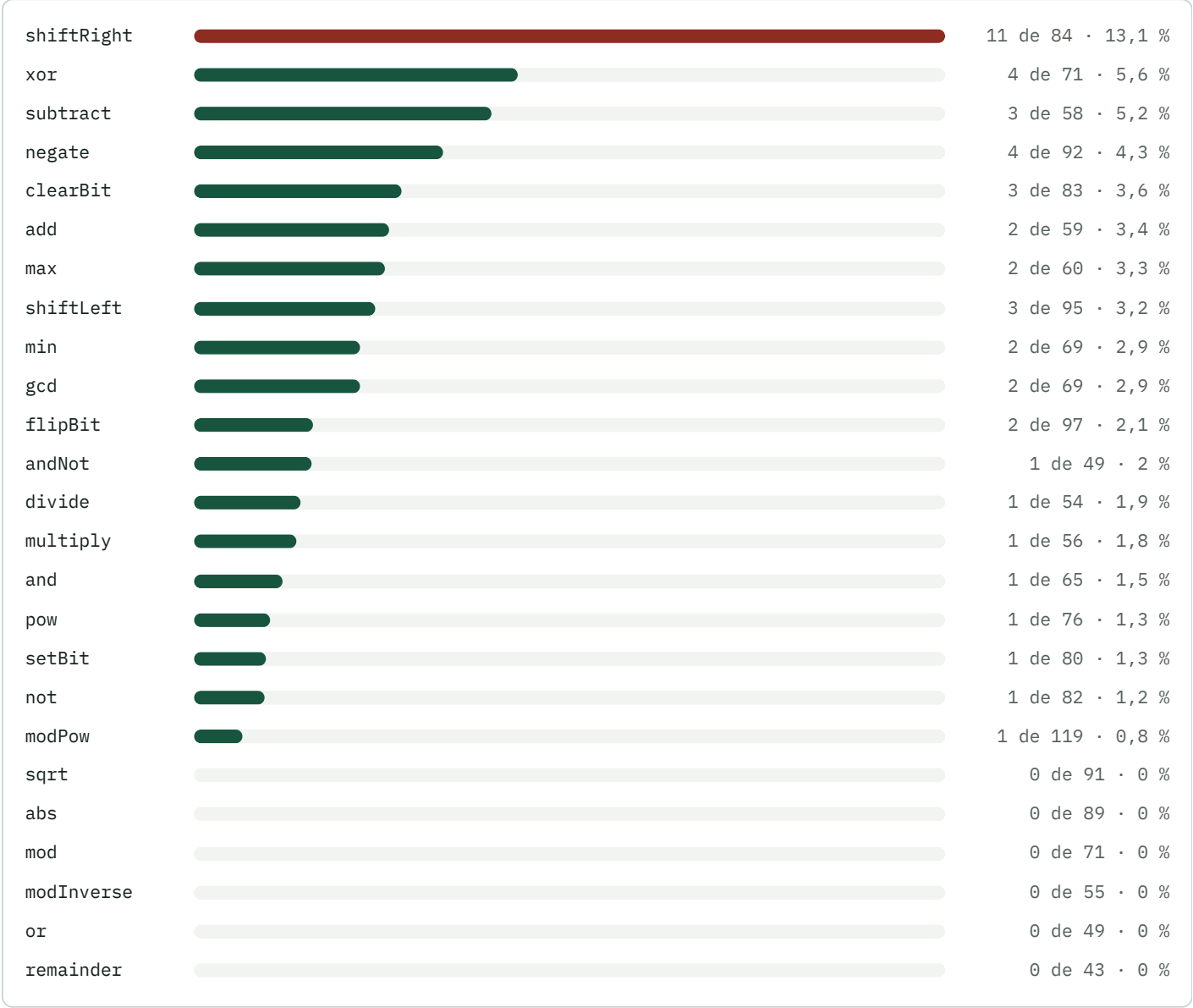
Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 532 a 103 caracteres en 7 pasos

Dónde se concentran las divergencias

Para cada operación, en cuántas de las expresiones que la usan hubo divergencia. Que una operación aparezca en una expresión que diverge no prueba que sea la culpable: eso lo dicen los

casos mínimos.



Todas las divergencias

Cada expresión en la que OpenJDK y KajiJDK dieron un resultado distinto, en el orden en que aparecieron.

#	TIPO	EXPRESIÓN GENERADA	OPENJDK	KAJIJDK
1	excepción contra valor	String.valueOf(new BigInteger("3822933037875768063649085183642879854").modPow(new BigInteger("85687768007531051271327124").modPow(new BigInteger("330478879430472332414269"), new BigInteger("340282366920938463463374607431768211456")).flipBit(1), new BigInteger("-189279246255402").shiftRight(31).modPow(new BigInteger("4811923206227507985").negate(), new BigInteger("9031773851424641005820520004304797591325152537675991331314").modPow(new BigInteger("-9223372036854775808"), new BigInteger("484412637231616293449897")))).longValue())	-5642153257516419978	EXC:java.lang.ArithmeticException
2	otro valor	String.valueOf(new BigInteger("-2").clearBit(31).andNot(new BigInteger("1962795419463368916").pow(2)).xor(new BigInteger("256")).testBit(31))	false	true
3	otro valor	String.valueOf(new BigInteger("-1").shiftRight(33).longValue())	-1	0

#	TIPO	EXPRESIÓN GENERADA	OPENJDK	KAJIJDK
4	otro valor	<code>String.valueOf(new BigInteger("-2147483648").shiftRight(32).getLowestSetBit())</code>	0	-1
5	otro valor	<code>new BigInteger("-98765432109876543210987654321").shiftRight(31).multiply(new BigInteger("340282366920938463463374607431768211456")).negate().max(new BigInteger("10").clearBit(32)).toString(8)</code>	-61672210063577733044164	-61672210063577733044152
6	otro valor	<code>new BigInteger("-18446744073709551616").min(new BigInteger("6465398639695")).and(new BigInteger("18446744073709551615")).flipBit(64).gcd(new BigInteger("-2").shiftRight(64)).toString()</code>	0	18446744073709551616
7	otro valor	<code>String.valueOf(new BigInteger("-407584303955795180166036188761314192023918").testBit(31))</code>	false	true
8	otro valor	<code>String.valueOf(new BigInteger("256").negate().testBit(63))</code>	true	false
9	otro valor	<code>new BigInteger("-390136690303871613558547759").shiftRight(64).shiftLeft(0).xor(new BigInteger("-18446744073709551616").subtract(new BigInteger("0")).divide(new BigInteger("-18446744073709551616").max(new BigInteger("10")))).toString()</code>	1844674407358476080	1844674407358476095
10	otro valor	<code>String.valueOf(new BigInteger("-18446744073709551616").add(new BigInteger("18446744073709551616").not()).shiftRight(65).longValue())</code>	-2	-1
11	otro valor	<code>String.valueOf(new BigInteger("-65965822096338883813426810717").testBit(64))</code>	false	true
12	otro valor	<code>new BigInteger("-1").xor(new BigInteger("9223372036854775808")).subtract(new BigInteger("-368085817248671590511407942005123312455").shiftRight(32)).toString()</code>	85701657740645462007791682427	85701657740645462007791682426
13	otro valor	<code>String.valueOf(new BigInteger("18446744073709551615").negate().setBit(63).shiftLeft(-1).intValue())</code>	0	1
14	otro valor	<code>new BigInteger("6376482070658812300672125607").gcd(new BigInteger("2").shiftLeft(65).add(new BigInteger("-40731248420276610866911063409357836402191615").subtract(new BigInteger("9435496438862"))).shiftRight(100)).toString()</code>	36893456016129603630	36893456016129603631
15	otro valor	<code>new BigInteger("-2").shiftRight(32).min(new BigInteger("123456789012345678901234567890")).toString()</code>	-1	0
16	otro valor	<code>new BigInteger("4294967296").xor(new BigInteger("-4294967296")).shiftRight(65).clearBit(31).toString()</code>	-2147483649	0

Cómo se obtuvo

- **Generar.** 6 lotes de 120 expresiones de `BigInteger`, de hasta tres niveles de operaciones anidadas. Las constantes salen, seis de cada diez veces, de una lista de bordes: cero, uno, menos uno, los límites de 32 y 64 bits y números de cientos de dígitos.
- **Ejecutar.** Cada lote es un único programa Java, compilado una sola vez con el `javac` de Eclipse Adoptium 25.0.4.1. El mismo `.class` corre en ese JDK y en KajiJDK con `KajiLibrary`.
- **Comparar.** Dos resultados coinciden si su texto es idéntico. Cuando una expresión lanza una excepción se compara la clase de la excepción, no el mensaje, que puede variar entre implementaciones sin estar mal.
- **Reducir.** De cada tipo de divergencia distinto se toma la primera, y se prueban versiones más chicas: quitar una operación, cambiar una constante por 0, 1, -1 o 2, acortar sus dígitos. Se queda con la más chica que todavía diverge del mismo modo, y repite.
- **Reproducir.** La misma semilla genera las mismas expresiones: `python test-fuzzing/fuzzer/bigfuzz.py --jdk "$OPENJDK" --seed 1 --lotes 6 --por-lote 120`.

Dónde sirve

- **Reescrituras y migraciones.** La versión vieja hace de oráculo, así que no hay que escribir el resultado esperado de cada caso.
- **Versión contra versión.** La release de hoy contra la de ayer, sobre las mismas entradas generadas.
- **Código numérico.** Redondeos, desbordes, divisiones, fechas y coma flotante: donde viven los bordes.
- **Parsers y serializadores.** Lo que se codifica y se decodifica tiene que volver igual.
- **Dos implementaciones de una misma especificación.** Cualquier diferencia es un error de una de las dos, como KajiLibrary y OpenJDK en este informe.

Lo que no resuelve

- **Necesita una referencia.** Una implementación confiable, o una propiedad que valga siempre, como que decodificar lo codificado devuelva la entrada.
- **Necesita determinismo.** Fecha, hora y todo lo aleatorio tienen que quedar fijos, o el oráculo reporta ruido.
- **Encuentra diferencias, no culpables.** Qué versión tiene razón lo decide una persona con la especificación en la mano.
- **Hay que auditar el instrumento.** Un oráculo roto, por ejemplo uno que compare una implementación consigo misma, reporta cero divergencias aunque haya errores.

Datos de 17 de septiembre de 2026 a las 12:29 p. m.. KajiJDK compilado el 17 de septiembre de 2026 a las 11:59 a. m.. Tiempo de ejecución: 1 s en OpenJDK y 250 s en KajiJDK, sin contar la reducción.